

Advanced SQL Operations

Comprehensive Notes for Database Querying & Optimization

1. Complex Joins

DEFINITION

Joins are used to combine rows from two or more tables based on a related column between them. "Complex Joins" refer to operations beyond the standard INNER JOIN, such as SELF JOINS (joining a table to itself to represent hierarchies), FULL OUTER JOINS (returning all records when there is a match in either left or right table), and CROSS JOINS (producing a Cartesian product).

QUERY SYNTAX (FULL OUTER JOIN)

```
SELECT a.column_name, b.column_name
FROM table1 a
FULL OUTER JOIN table2 b
  ON a.id = b.id;
```

EXAMPLE (SELF JOIN FOR HIERARCHICAL DATA)

A common scenario in enterprise applications is finding which manager supervises which employee within the same `Employees` table.

```
SELECT
  e.EmployeeName AS Employee,
  m.EmployeeName AS Manager
FROM
  Employees e
LEFT JOIN
  Employees m ON e.ManagerID = m.EmployeeID;
```

This query links the `ManagerID` of an employee to the `EmployeeID` of their manager within the exact same table.

2. Subqueries

DEFINITION

A subquery (or inner query) is a query nested inside another query (such as SELECT, INSERT, UPDATE, or DELETE). Subqueries can return individual values, single lists, or entire records. A **Correlated Subquery** evaluates once for every row processed by the outer query, making it powerful but potentially resource-intensive.

QUERY SYNTAX

```
SELECT column_name
FROM table_name
WHERE column_name operator (
    SELECT column_name
    FROM table_name
    WHERE condition
);
```

EXAMPLE (FILTERING AGAINST AN AGGREGATED VALUE)

Finding employees who earn more than the average salary across the entire company.

```
SELECT
    EmployeeName,
    Department,
    Salary
FROM
    Employees
WHERE
    Salary > (SELECT AVG(Salary) FROM Employees);
```

The inner query calculates the company-wide average salary first. The outer query then retrieves only the records where the salary exceeds that scalar result.

3. Window Functions

DEFINITION

Window functions perform a calculation across a set of table rows that are somehow related to the current row. Unlike aggregate functions with `GROUP BY`, window functions do not cause rows to become grouped into a single output row — the rows retain their separate identities.

QUERY SYNTAX

```
window_function_name (expression) OVER (  
    [PARTITION BY partition_expression]  
    [ORDER BY sort_expression]  
)
```

EXAMPLE (RANKING ITEMS PER CATEGORY)

Ranking the highest-paid employees within their specific departments.

```
SELECT  
    EmployeeName,  
    Department,  
    Salary,  
    RANK() OVER (PARTITION BY Department ORDER BY Salary DESC) as  
    DeptSalaryRank  
FROM  
    Employees;
```

`PARTITION BY` divides the result set into distinct departments. `ORDER BY` sorts salaries in descending order within those partitions. `RANK()` applies the ranking value to each row independently.

4. Common Table Expressions (CTEs)

DEFINITION

A Common Table Expression (CTE) provides a temporary result set that you can reference within a SELECT, INSERT, UPDATE, or DELETE statement. CTEs improve readability and organization of complex queries by breaking them down into logical blocks. They also support recursive queries.

QUERY SYNTAX

```
WITH CteName (Column1, Column2) AS (  
    -- CTE Query Definition  
    SELECT Column1, Column2  
    FROM TableName  
)  
-- Main Query  
SELECT * FROM CteName;
```

EXAMPLE (SIMPLIFYING A COMPLEX AGGREGATION)

Calculating the average total sales per user without using complex nested subqueries in the FROM clause.

```
WITH UserTotalSales AS (  
    SELECT  
        UserID,  
        SUM(OrderTotal) as TotalSales  
    FROM  
        Orders  
    GROUP BY  
        UserID  
)  
SELECT  
    AVG(TotalSales) AS AverageUserSpend  
FROM  
    UserTotalSales;
```

The CTE `UserTotalSales` aggregates all orders by user first. The main query then elegantly calculates the average of those totals, drastically improving the query's readability.